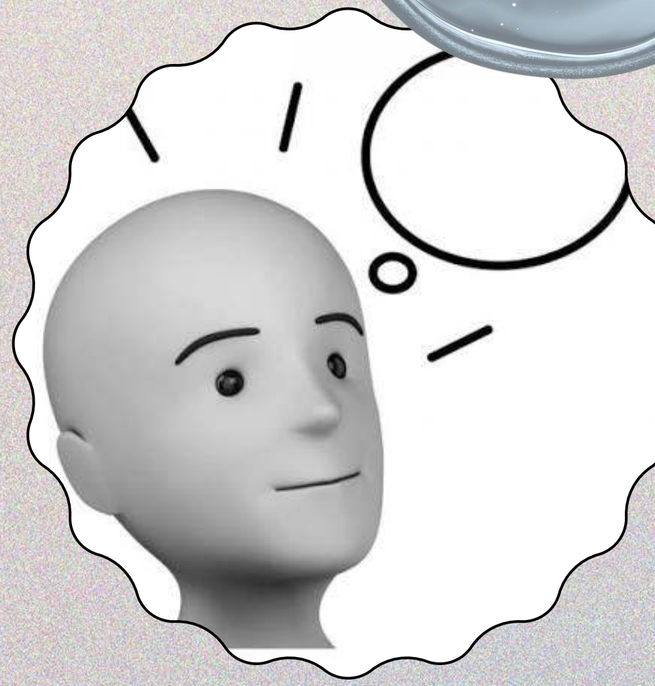
APUNTES JUEVES 29 DE OCTUBRE

COMPILADORES E INTÉRPRETES

IAN SEBASTIAN UGALDE MUÑOZ
2024124076

RECORDEMOS

1. PREPROCESO
2. FIRST
3. FOLLOW
4. PREDICT
5. TABLA



Preproceso-> que habíamos hecho anteriormente: lo de eliminar la recursividad por la izquierda

FACTORIZACIÓN POR LA IZQUIERDA

si hay dos o más reglas de la forma

A-> αβ

A → αγ

EXPLICACIÓN:

si tiene ese formato no nos sirve

Hay que curarlo, ¿Cómo? (nidea (no es como que el titulo no de la pista))

La hilera a es lo más larga posible

Entonces las cambiamos por:

A-> αA’

A’-> β|γ

el chiste es claramente factorizar lo de la izquierda



EJEMPLO 1



SELECTED WORKS

SQ → S; SQ
SQ → S E
S → <statement>

SQ → SSQ’
SQ' → ; SQ → es un | or
SQ' → E
S → <statement>

EJEMPLO 2



SELECTED WORKS

IFS → if(E) S

IFS → if (E)S else S

entonces:

IFS → if(E) S IFS’

IFS’ → ε | else S

De nuevo, Preproceso queda:

1. Quitar recursividad por la izquierda
2. Factorizar por la izquierda

ANÁLISIS SINTÁCTICO

FIRST







CÁLCULO DE FIRST

Sea G una CFG
La función FIRST(X) regresa un conjunto de terminales y posiblemente ε que indican todos los “inicios” factibles de X bajo la gramatica G.

IMAGINEMONOS A BACKUS

CÁLCULEMOS FIRST()
FIRST(X)

el Argumento X puede ser

1. un terminal
2. un No- terminal
3. Una hilera de terminales y no terminales (puede ser ε)

CÁLCULO DE FIRST-CONT

CASOS TRIVIALES DE FIRST (X)

Si X es ε se regresa el conjunto {ε}

Si X es un terminal se regresa el conjunto que contenga este terminal :
FIRST(+)= {+}
FIRST(id)= {id}

CÁLCULEMOS FIRST() FIRST(X)

DATAZOS DE LA CLASE:

3 ACTORES:
ADRIANA
TORRES
LOS PERROS



Podriamos decir:
que torres le tiraba (un palo aΓ | εβ) los perros
(β->a
Γ-> de) adriana

FIRST(X) DE UN NO-TERMINAL

X ES UN NO TERMINAL

Por cada regla de la gramatica X → X_1, X_2, X_3 .... X_n haga:
Una (de unir jaja) (FIRST(X_1) - {ε}) a FIRST(X)

Nota: X podría empezar con X_2 si X_1 contiene ε

Osea:
Si FIRST(X_1) contiene a ε:
Una (FIRST(X_2) - {ε}) a FIRST(X)
Y si Si FIRST(X_2) contiene a ε:
Una (FIRST(X_3) - {ε}) a FIRST(X)
Y SI Si FIRST(X_3) contiene a ε:
Una (FIRST(X_4) - {ε}) a FIRST(X)

si ε pertenece a FIRST(X_1), FIRST(X_2), FIRST(X_n) ENTONCES una **{ε} a FIRST(X)**

y REPITA hasta que no haya cambios

FIRST(X) DE UNA HILERA

X ES UNA HILERA X_1, X_2, X_3 .... X_N

Haga:
Si FIRST(X_1) contiene a ε:
Una (FIRST(X_2) - {ε}) a FIRST(X)
Y si Si FIRST(X_2) contiene a ε:
Una (FIRST(X_3) - {ε}) a FIRST(X)
Y SI Si FIRST(X_3) contiene a ε:
Una (FIRST(X_4) - {ε}) a FIRST(X)

si ε pertenece a FIRST(X_1), FIRST(X_2), FIRST(X_n) ENTONCES una **{ε} a FIRST(X)**

EJEMPO 1 DE FIRST()

ANTES DE EMPEZAR:
Ojo necesitamos que nos den todas nuestras reglitas

E-> E OPT
E → T
OP-> +
OP-> -
T-> TMF
T-> F
M ->*
F → (E)
F → # (numeral es cualquier int)

Al inicio vamos a poner vacio ∅ el campito, y lo vamos a ir cambiando si corresponde

No terminal	Pasada 1	Pasada 2	Pasada 3
E	∅	∅	{(, #}
OP	{+}	{+, -}	{+, -}
T	∅	({, #)	({, #)
M	{*}	{*}	{*}
F	{(, #}	{ (, #}	{(, #}

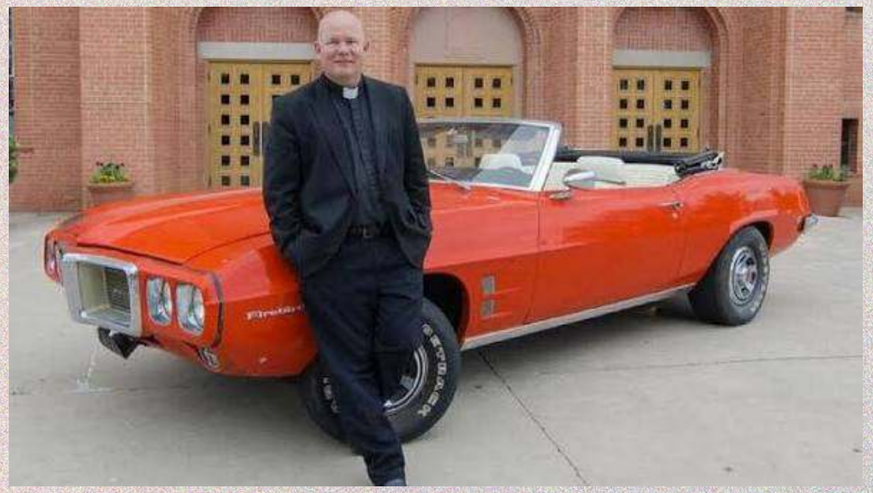
Existe una cuarta pasada implicita, pero no cambia nada, y entonces paramos

Pausa comercial

TORRES RECITANDO MISA

NO PREGUNTEN

- Pero la respuesta es que Torres se puso a recitar los siguientes slides como si estuviera dando Misa.
- Quién lo diria pero al parecer si ha ido, y lo más increíble: está bautizado.



El padrecito esperando a Torres para irse

EJEMPO 2 DE FIRST()

Al inicio vamos a poner vacio ∅ el campito, y
lo vamos a ir cambiando si corresponde

E-> TE’
E’ → OPT E’
E’ → ε
OP-> +
OP-> -
T-> FT’
T’-> MFT’
T’ → ε
M ->*
F → (E)
F → # (numeral es cualquier int)

No terminal	Pasada 1	Pasada 2	Pasada 3
E	**∅**	∅	**{ (, # }**
E‘	**{ ε }**	{ε, + - }	{ ε, + - }
OP	**{ +}**	**{ + , - }**	**{ + , - }**
T	**∅**	**({, #)**	**({, #)**
T’	**{ε}**	**{ε, *}**	**{ε, *}**
M	**{ * }**	**{ * }**	**{ * }**
F	**{ (, # }**	{ (, # }	**{ (, # }**

Existe una cuarta pasada implicita, pero no
cambia nada, y entonces paramos

EJEMPO 3 DE FIRST()

S → aSe
S-> B
B-> bBe
B-> C
C-> cCe
C-> d

Al inicio vamos a poner vacio ∅ el campito, y lo vamos a ir cambiando si corresponde

No terminal	Pasada 1	Pasada 2
s	{a}	{a,b,c,d}
B	**{ a,b }**	{b,c,d}
C	**{ c, d}**	{c,d}

Existe una tercera pasada implicita, pero no cambia nada, y entonces paramos

EJEMPO 4 DE FIRST()

reglas

S → ABc
A → a
B-> b
A-> ε
B-> ε

Al inicio vamos a poner vacio ∅ el campito, y lo vamos a ir cambiando si corresponde

No terminal	Pasada 1	Pasada 2	Pasada 3
s	∅	{a}	**{a,b,c}**
B	**{ a }**	{a, ε}	{a, ε}
C	{b, ε}	{b, ε}	{b, ε}

Existe una cuarta pasada implicita, pero no cambia nada, y entonces paramos

CÁLCULO DE FOLLOW

Sea G una CFG
La función FOLLOW(X) regresa un conjunto de terminales y posiblemente $ que indican todas las “continuaciones” factibles del No terminal X bajo la gramática G

El Argumento X solo puede ser un No-terminal
El calculo de FOLLOW es iterativo hasta que no haya más cambios

CÁLCULO DE FOLLOW

FOLLOW (X)

X es un no terminal

Reglas :

1. Si X es el simbolo inicial, agregue $ a FOLLOW(X)
2. Si hay una regla → aXβ agruegue FIRST(β) -{ε} a FOLLOW(X)
3. Si hay una regla A→ aXβ y ε pertenece a FIRST(β) agregue FIRST (A) a FOLLOW(β)

Observaciones de Folow:

- Solo se calcula para no terminales
- ε nunca es parte de FOLLOW() pero $ si puede estar.
- Primero hay que calcular todos los FIRST() de los No-Terminales
- Se ignoran llas reglas de la gramática que únicamente tengan terminales o ε en su lado derecho, los demás se procesan repetidamente hasta no haber cambios

EJEMPLO 1 DE FOLLOW

E-> E OPT
E → T
OP -> +
OP-> - X
T-> TMF
T-> F
M ->* X
F → (E)
F → # X

No terminal	FIRST	Pasada 1	Pasada 2
E	**{ (, # }**	**{ $, +. - }**	
OP	**{ + , - }**	**{ (, #}**	
T	**({, #)**	{$, +, -, *}	
M	**{ * }**	{(, #}	
F	**{ (, # }**	**{$, +, -, *}**	

Agregamos $, por ser el simbolo de entrada a E, y +, - por ser el FIRST(OP)

follow(OP) le agregamos el First(T)

Folllow(T) es agregar el FIRST de M, Pero se le agrega el FOLLOW de E también por E->EOPTε
o incluso E-> Tε

A follow de M se agrega el First de F

y a Folow de F se agrega el FIRST de T por T->TMFε

EJEMPLO 1 DE FOLLOW

E-> E OPT
E → T
~~OP -> +~~
~~OP-> -~~ X
T-> TMF
T-> F
~~M ->*~~X
F → (E)
~~F → #~~ X

No terminal	FIRST	Pasada 1	Pasada 2
E	**{ (, # }**	**{ $, +. - }**	
OP	**{ + , - }**	**{ (, #}**	
T	**({, #)**	{$, +, -, *}	{$, +, -, *,)}
M	**{ * }**	{(, #, *}	
F	**{ (, # }**	**{$, +, -, *}**	**{$, +, -, *,)}**

Bueno y así se sigue, es un poco confuso, así que mucho cuidado

EJEMPLO 2 DE FOLLOW

vamos con la primera pasada, calculamos el follow de cada no terminal

E-> TE’
E’ → OPT E’
~~E’ → ε~~
~~OP-> +~~
~~OP ->~~
T-> FT’
T’-> MFT’
~~T’ → ε~~
~~M ->*~~
F → (E)
~~F → #~~

No terminal	FIRST	PASADA 1	PASADA 2
E	**{ (, # }**	**{$,)}**	
E‘	{ ε, + , - }	{$}	
OP	**{ + , - }**	**{(, #}**	
T	**{(, #}**	**{+, - , $, * }**	
T’	**{ε, *}**	**{+, - , $ }**	
M	**{ * }**	**{(, #}**	
F	**{ (, # }**	**{*, +,- , $}**	

Ya saben inicial, $ y) que es el follow

Follow de OP es Fiirst(T)- ε

Follow de T es Fiirst(E’)- ε + EL follow(E’)

Follow de T’ agregue FOLLOW (T)

Follow de M agregue FIRST (F)

FOLLOW(F) agregue FIRST (T’)) Y el follow(T) por T-> FT’ ya que contiene epsilón

EJEMPLO 2 DE FOLLOW

E-> TE’
E’ → OPT E’
~~E’ → ε~~
~~OP-> +~~
~~OP ->~~
T-> FT’
T’-> MFT’
~~T’ → ε~~
~~M ->*~~
F → (E)
~~F → #~~

No terminal	FIRST	PASADA 1	PASADA 2
E	**{ (, # }**	**{$,)}**	
E‘	{ ε, + , - }	{$}	{$,)}
OP	**{ + , - }**	**{(, #}**	
T	**{(, #}**	**{+, - , $ }**	**{+, - , $,)}**
T’	**{ε, *}**	**{+, - , $ }**	**{+, - , $,)}**
M	**{ * }**	**{(, #}**	
F	**{ (, # }**	**{*, +,- , $}**	**{*, +,- , $,(,)}**

agregamos lo que nos faltaba, y al final no ocupamos otra pasada

THANK YOU

PRESENTED BY : IAN UGALDE MUÑOZ

2024124076